

KKO PROJEKT - Komprese obrazových dat slovníkovými metodami

Projekt: KKO PROJEKT - Komprese obrazových dat slovníkovými metodami

Autor: Dominik Honza

Login: xhonza04

Úvod do problematiky

Cílem projektu je bezztrátová komprese 8bitových RAW obrazů, kde samotná obrazová data neobsahují žádná metadata o rozměrech ani o způsobu zpracování.

Kompresní program musí kromě samotného zakódování dat zajistit také uložení všech informací potřebných pro jejich pozdější správnou rekonstrukci. Dekomprese proto nesmí záviset na ručním zadávání parametrů, které byly známy pouze v okamžiku komprese. Součástí řešení je tedy nejen návrh kompresního algoritmu, ale také návrh vhodné hlavičky souboru a přehledné struktury programu.

Vybraná metoda

Jako kompresní metoda byla zvolena LZSS, protože jde o bezztrátovou slovníkovou metodu založenou na vyhledávání opakovaných sekvencí ve vstupních datech. Její výhodou je poměrně přímočará implementace, dobrá srozumitelnost a zároveň možnost dalšího rozšíření, například o efektivnější vyhledávání shod nebo jinou organizaci slovníku. Pro tento projekt je vhodná také tím, že umožňuje dobře oddělit vlastní kompresní mechanismus od ostatních částí programu, jako je práce s hlavičkou souboru nebo předzpracování dat.

Nad touto metodou je použit volitelný diferenční model, který před vlastní kompresí převádí data na rozdíly mezi po sobě jdoucími hodnotami. Tento krok může zlepšit lokální regularitu vstupu a tím i úspěšnost následné komprese. Obrazová data se často liší vedle sebe jen málo a tvoří gradienty, což vytváří opakující se vzorce nebo podobné posloupnosti, které lze snadno a efektivně komprimovat.

V adaptivním režimu je obraz rozdělen na bloky a pro každý blok se porovnává horizontální a vertikální skenování. Vybere se ta varianta, která vede k menšímu výstupu. Součástí návrhu je také slovníková vrstva pro vyhledávání shod v LZSS. Ta je variantou využívající hash tabulku, kde se do ní ukládají pozice řetězců začínajících na daném místě ve slovníku. Klíčem je hash hodnota vypočtená z několika po sobě jdoucích znaků nebo bajtů. Při zpracování nové pozice vstupu se stejným způsobem vypočte hash aktuální sekvence a v hash tabulce se vyhledají kandidátní pozice se shodnou hodnotou. Následně se na těchto pozicích provede přímé porovnání délky skutečné shody. Vybere se nejdelší nalezená shoda, kterou lze zakódovat jako odkaz do slovníku; pokud není shoda dostatečně dlouhá, uloží se symbol přímo jako literál. Po posunu v datech se slovník i hash tabulka průběžně aktualizují, aby odpovídaly aktuálnímu obsahu posuvného okna.

Struktura projektu

Projekt je rozdělen do samostatných částí pro lepší rozdělení zodpovědností a případné rozšíření:

1. **main** a zpracování argumentů: **main** pouze načte argumenty příkazové řádky a rozhodne, zda se spustí komprese nebo dekomprese. Tím zůstává vstupní bod programu velmi jednoduchý a řídicí logika se nerozptyluje do detailů implementace.

2. **Compressor** a **Decompressor**: tyto dvě třídy představují hlavní orchestrační vrstvu. Starají se o otevření souborů, základní error handling a řízení celého workflow podle zvoleného režimu. Vykonávání začíná jednotným společným bodem metodou **execute**, která je zodpovědná za předchystání souborů, případnou kontrolu chyb a samotné spuštění dekomprese nebo komprese.
3. **ConfigProvider**: centralizuje výchozí konfiguraci a její validaci. Při kompresi kontroluje, zda rozměry i parametry odpovídají požadavkům formátu a zvoleného režimu. V případě neshod dimenzí nebo jiných chyb nedovolí běh programu. Umožňuje jednoduchou úpravu konfiguračních parametrů komprese.
4. **HeaderProvider**: zajišťuje zápis a čtení hlavičky komprimovaného souboru. Jedná se o uniformní rozhraní pro práci s hlavičkou v obou směrech dekomprese i komprese. Při kompresi ukládá do hlavičky všechny informace potřebné pro pozdější rekonstrukci dat, při dekompresi je naopak načte a vyplní podle nich konfiguraci dekodéru.
5. **LZSS**: obsahuje vlastní implementaci kompresní a dekompresní metody. Tato část řeší práci s historií, hledání shod a převod dat mezi surovou a zakódovanou podobou.
6. **ByteIO** a utility: pomocné třídy sjednocují zápis a čtení binárních hodnot do souboru i do paměťových bufferů. Díky tomu jsou low-level I/O operace soustředěné na jednom místě a zbytek projektu se může soustředit na algoritmickou logiku. Napomáhá to také k požadované přenositelnosti uvedené v zadání.

Workflow komprese a dekomprese

Workflow komprese

Při spuštění s přepínačem **-c** program nejprve zpracuje argumenty **-i**, **-o**, **-w** a volitelně **-m** a **-a**. Následně **Compressor** otevře vstupní a výstupní soubor, načte výchozí konfiguraci z **ConfigProvider** a provede validaci vstupu. Potom připraví hlavičku obsahující rozměry obrazu, původní velikost, aktivní přepínače a parametry LZSS a zapíše ji pomocí **HeaderProvider**.

Teprve poté začíná samotná komprese dat. Pokud není aktivní adaptivní režim, zpracovává se celý obraz jako jeden souvislý blok. Pokud je zapnut přepínač **-a**, obraz se rozdělí na bloky a pro každý blok se vyzkouší horizontální i vertikální pořadí čtení. Před vlastní kompresí se podle přepínače **-m** může aplikovat diferenční model. Nakonec se nad připravenými daty spustí LZSS a do výstupu se uloží buď komprimovaná data, nebo původní blok, pokud by komprese nevyšla výhodněji.

Workflow dekomprese

Při spuštění s přepínačem **-d** program nepotřebuje parametry **-w**, **-m** ani **-a**, protože všechny potřebné informace jsou uloženy v hlavičce souboru. **Decompressor** otevře vstup, pomocí **HeaderProvider** načte hlavičku a z ní obnoví konfiguraci i vlastnosti původních dat. Tím je zajištěno, že dekomprese probíhá se stejným nastavením, s jakým byla provedena komprese.

Pak se zpracovává samotný datový obsah. Ve statickém režimu se čtou jednotlivé bloky za sebou, případně se nad nimi spustí dekomprese LZSS a následně se aplikuje inverzní diferenční model. V adaptivním režimu se navíc podle příznaků u každého bloku obnoví správné horizontální nebo vertikální uspořádání a bloky se skládají zpět do výstupního rastru. Výsledkem je přesná rekonstrukce původního RAW obrazu.

Výsledky vyhodnocení výkonnosti

File	Mode	Orig (B)	Comp (B)	Ratio	BPP	Entropy	C Time	D Time	Status
cb.raw	base	262144	3342	0,0127	0,1020	1,0024	,0335	,0175	OK
cb.raw	-m	262144	3440	0,0131	0,1050	1,0024	,0386	,0128	OK
cb.raw	-a	262144	3611	0,0138	0,1102	1,0024	,0794	,0120	OK
cb.raw	-a_-m	262144	3675	0,0140	0,1122	1,0024	,1039	,0137	OK
cb2.raw	base	262144	9659	0,0368	0,2948	6,9060	,0334	,0119	OK
cb2.raw	-m	262144	9337	0,0356	0,2849	6,9060	,1510	,0143	OK
cb2.raw	-a	262144	10731	0,0409	0,3275	6,9060	,0765	,0227	OK
cb2.raw	-a_-m	262144	9367	0,0357	0,2859	6,9060	,2347	,0134	OK
df1h.raw	base	262144	3537	0,0135	0,1079	8,0000	,0256	,0100	OK
df1h.raw	-m	262144	3253	0,0124	0,0993	8,0000	,0306	,0130	OK
df1h.raw	-a	262144	5675	0,0216	0,1732	8,0000	,0504	,0152	OK
df1h.raw	-a_-m	262144	3467	0,0132	0,1058	8,0000	,0766	,0136	OK
df1hvx.raw	base	262144	16312	0,0622	0,4978	3,2547	,0899	,0118	OK
df1hvx.raw	-m	262144	18540	0,0707	0,5658	3,2547	,2949	,0145	OK
df1hvx.raw	-a	262144	19154	0,0731	0,5845	3,2547	,1471	,0153	OK
df1hvx.raw	-a_-m	262144	21737	0,0829	0,6634	3,2547	,4019	,0136	OK
df1v.raw	base	262144	4388	0,0167	0,1339	0,4238	,0288	,0121	OK
df1v.raw	-m	262144	3253	0,0124	0,0993	0,4238	,0984	,0122	OK
df1v.raw	-a	262144	4523	0,0173	0,1380	0,4238	,0558	,0139	OK
df1v.raw	-a_-m	262144	3479	0,0133	0,1062	0,4238	,0862	,0132	OK
nk01.raw	base	262144	262180	1,0001	8,0011	6,4650	,2523	,0112	OK
nk01.raw	-m	262144	262180	1,0001	8,0011	6,4650	,2316	,0104	OK
nk01.raw	-a	262144	262315	1,0007	8,0052	6,4650	,3222	,0132	OK
nk01.raw	-a_-m	262144	262315	1,0007	8,0052	6,4650	,3044	,0195	OK
shp.raw	base	262144	4969	0,0190	0,1516	0,1824	,0710	,0123	OK
shp.raw	-m	262144	5171	0,0197	0,1578	0,1824	,0550	,0122	OK
shp.raw	-a	262144	4395	0,0168	0,1341	0,1824	,1375	,0122	OK
shp.raw	-a_-m	262144	4491	0,0171	0,1371	0,1824	,1314	,0253	OK
shp1.raw	base	262144	47928	0,1828	1,4626	1,8943	,2765	,0227	OK

File	Mode	Orig (B)	Comp (B)	Ratio	BPP	Entropy	C Time	D Time	Status
shp1.raw	-m	262144	51498	0,1964	1,5716	1,8943	,1492	,0130	OK
shp1.raw	-a	262144	30974	0,1182	0,9453	1,8943	,4678	,0127	OK
shp1.raw	-a_-m	262144	32760	0,1250	0,9998	1,8943	,4182	,0150	OK
shp2.raw	base	262144	59166	0,2257	1,8056	1,9007	,3322	,0199	OK
shp2.raw	-m	262144	64140	0,2447	1,9574	1,9007	,2097	,0162	OK
shp2.raw	-a	262144	51722	0,1973	1,5784	1,9007	,6404	,0139	OK
shp2.raw	-a_-m	262144	56053	0,2138	1,7106	1,9007	,4630	,0199	OK

Použité zdroje

1. STOER, J. A., a SZYMAŃSKI, T. G. Data compression via textual substitution. *Journal of the ACM*, 1982, 29(4), 928–951.
2. SALOMON, D. *Data Compression: The Complete Reference*. 3rd ed. New York: Springer, 2004.
3. Lempel–Ziv–Storer–Szymanski (LZSS). In: Wikipedia: the free encyclopedia [online]. Dostupné z: <https://en.wikipedia.org/wiki/Lempel%E2%80%93Ziv%E2%80%93Storer%E2%80%93Szymanski>